

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Siva Charan	Reg. No.:	192525039
Section / Batch:	2025	Date:	26/08/2026

Code of Conduct

I, Siva Charan (192525039) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Siva Charan

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

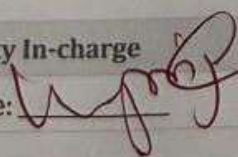
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Linear Probing vs Separate Chaining

<u>Factor</u>	<u>Linear Probing</u>	<u>Separate Chaining</u>
Clustering	Causes Primary Clustering	Less Clustering
Memory	uses less memory	Requires extra memory for linked lists.
High load factor	Performance decreases quickly.	Performing better at high load factor.
Large-scale use	Suitable for smaller tables.	Better for large-scale applications.

Conclusion : Separate Chaining is preferred for larger applications.

2) Quick Sort vs Merge Sort for Linked List

<u>Factor</u>	<u>Quick Sort</u>	<u>Merge Sort</u>
Time	Average $O(n \log n)$, worst $O(n^2)$	$O(n \log n)$
Memory	$O(\log n)$ average stack	$O(\log n)$ recursion
Cache	Less suitable for linked list	Better sequential access
Linked List	Less suitable	Highly suitable

3) Quadratic Probing vs double Hashing

<u>Factor</u>	<u>Quadratic Probing</u>	<u>Double Hashing</u>
Clustering	Reduces Primary Clustering but may have Secondary Clustering.	Reduces both Clustering types.
Efficiency	Good at low load factor.	Generally more efficient.
Dense Tables	Less suitable.	More suitable.

4) Best-Case vs worst-Case Sorting Complexity

<u>Algorithm</u>	<u>Best</u>	<u>Worst</u>
Bubble Sort	$O(n)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$

Practical Significance : worst-case complexity is important when predictable performance is required.

Recursive vs Iterative Quick Sort

Factor

Recursive

Iterative

Space

uses function call stack,
usually $O(\log n)$

uses an explicit stack,
usually $O(\log n)$

Implementation

Easier and Shorter

More complex

Stack overflow

Higher risk with un-
balanced partitioning

Lower system-stack
risk

Control

Less control over stack

Better control

Conclusion: Recursive Quick Sort is easier to
implement.

